

I N D E X

Name Aditi Rastogi

Class 4A

Roll No 18F24CS017 Subject OS LAB

School _____

Sl No.	Date	Title	Page No.	Teacher Sign/Remarks
1.	24/02/26	LAB 0 - Some basic C programmes	1-5	} 8/2 (10)
2.	03/03/26	LAB 1 - FCFS & SJF algorithm	6-12	
3.	10/03/26	LAB 2 - Priority Scheduling		(10) 8/2
4.	17/03/26	LAB 3 - Round Robin	(9)	8/2 10/3
5.	24/03/26	LAB 4 - Multi Level Queue	(9)	8/2 17/3
6.	31/03/26	LAB 5 - RMS & EDF RTOS	(8)	8/2 24/3
7.	21/04/26	LAB 6 - Bounded Buffer & Philosopher Problem	(10)	8/2 31/3 21/4
8.	28/04/26	LAB 7 - Banker's Safety Algorithm	(10)	8/2 28/4
9.	05/05/26	LAB 8 - DEADLOCK Detection		8/2 5/5
10.	12/05/26	LAB 9 - Memory Allocation	10	8/2
11.	26/05/26	LAB 10 - Page replacement Algorithm		2/5

WAP to

Q → Find the Second Smallest and Second Largest element in an array

#include <stdio.h>

void SSL (int arr[], int n) {

for (int i=0; i<n-1; i++) {

int minindex = i;

for (int j=i+1; j<n; j++) {

if (arr[j] < arr[minindex]) {

minindex = j;

}

}

if (minindex != i) {

int temp = arr[i];

arr[i] = arr[minindex];

arr[minindex] = temp;

}

}

}

int main() {

int arr[5] = {5, 3, 1, 2, 4};

int n = sizeof(arr) / sizeof(arr[0]);

SSL (arr, n);

printf("Second Smallest Element is %d", arr[1]);

printf("Second Largest Element is %d", arr[n-2]);

}

• Output :-

Second Smallest Element is 2

Second Largest Element is 4

Q → WAP to find the sum of left diagonal elements of a matrix. ②

Sol:- #include <stdio.h>

```
int main() {
    int n, i, j;
    int sum = 0;
    printf("Enter the size of matrix (n x n): ");
    scanf("%d", &n);
    int matrix[n][n];
    printf("Enter the elements of matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }
    for (i = 0; i < n; i++) {
        sum += matrix[i][i];
    }
    printf("Sum of left diagonal = %d\n", sum);
    return 0;
}
```

• Output:-

Enter the size of matrix (n x n): 3

Enter the elements of matrix:

1 2 3

4 5 6

7 8 9

Sum of left diagonal = 15

Q → WAP to find the sum of rows and columns of a matrix. ③

Solⁿ:- #include <stdio.h>

```
int main() {
```

```
    int rows, cols, i, j;
```

```
    printf("Enter the number of rows: ");
```

```
    scanf("%d", &rows);
```

```
    printf("Enter the number of columns: ");
```

```
    scanf("%d", &cols);
```

```
    int matrix[rows][cols];
```

```
    printf("Enter the elements of matrix: \n");
```

```
    for(i=0; i<rows; i++){
```

```
        for(j=0; j<cols; j++){
```

```
            scanf("%d", &matrix[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\n Sum of each rows: \n");
```

```
    for(i=0; i<rows; i++){
```

```
        int rowSum = 0;
```

```
        for(j=0; j<cols; j++){
```

```
            rowSum += matrix[i][j];
```

```
        }
```

```
        printf("Row %d = %d \n", i+1, rowSum);
```

```
    }
```

```
    printf("\n Sum of each columns: \n");
```

```
    for(i=0; i<rows; i++){
```

```
        int colSum = 0;
```

```
        for(j=0; j<cols; j++){
```

```
            colSum += matrix[j][i];
```

```
        }
```

```
        printf("Column %d = %d \n", i+1, colSum);
```

```
    }
```

```
    return 0;
```

```
}
```


• Output:-

Enter number of rows: 3

Enter number of columns: 3

Enter the elements of the matrix:

1 2 3

4 5 6

7 8 9

Sum of each row:

Row 1 = 6

Row 2 = 15

Row 3 = 24

Sum of each column:

Column 1 = 12

Column 2 = 15

Column 3 = 18

Q → WAP to count the total number of duplicate elements in an array.

Solⁿ:- #include <stdio.h>

int main(){

int n, i, j, count = 0;

printf("Enter the size of the array:");

scanf("%d", &n);

int arr[n];

printf("Enter the elements of the array:\n");

for (i = 0; i < n; i++){

scanf("%d", &arr[i]);

}

for (i = 0; i < n; i++){

for (j = i + 1; j < n; j++){

if (arr[i] == arr[j]){

count++;

break;

}

}

}

printf("Total number of duplicate elements = %d\n", count);

return 0;

}

• Output:-

Enter the size of array: 8

Enter the elements of the array:

1 2 3 2 4 5 3 6

Total number of duplicate elements = 2.

Q → WAP to delete an element at a desired position from an array.

Sol:- #include <stdio.h>

int main() {

int n, pos, i;

printf("Enter the size of the array: ");

scanf("%d", &n);

int arr[n];

printf("Enter the elements of the array:");

for (i = 0; i < n; i++) {

scanf("%d", &arr[i]);

}

printf("Enter the position to delete (1 to %d):", n);

scanf("%d", &pos);

if (pos < 1 || pos > n) {

printf("Invalid position\n");

}

else {

for (i = pos - 1; i < n - 1; i++) {

arr[i] = arr[i + 1];

}

n--;

printf("Array after deletion: \n");

for (i = 0; i < n; i++) {

printf("%d ", arr[i]);

}
printf("\n");

}

}

• Output:-

Enter the size of the array: 5

Enter the elements of the array:

10 20 30 40 50

Enter the position to delete (1 to 5): 3

Array after deletion:

10 20 40 50

L101:-

03/03/26

• FCFS:-

```
#include <stdio.h>
```

```
void sent(int at[], int bt[], int pid[], int n) {
```

```
    for(int i=0; i<n-1; i++) {
```

```
        for(int j=0; j<n-i-1; j++) {
```

```
            if(at[j] > at[j+1]) {
```

```
                int temp = at[j];
```

```
                at[j] = at[j+1];
```

```
                at[j+1] = temp;
```

```
                temp = bt[j];
```

```
                bt[j] = bt[j+1];
```

```
                bt[j+1] = temp;
```

```
                temp = pid[j];
```

```
                pid[j] = pid[j+1];
```

```
                pid[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], ct[n], tat[n], wt[n], rt[n], pid[n], sumCT=0;
```

```
    float sumTAT=0, sumWT=0, sumRT=0;
```

```
    printf("Enter the arrival time and burst time for each  
    processes: \n");
```

```
    for(int i=0; i<n; i++) {
```

```
        pid[i] = i+1;
```

```
        printf("\n Arrival time for P%d: ", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst time for P%d: ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    sent(at, bt, pid, n);
```



```

for (int i = 0; i < n; i++) {
    sumCT += bt[i];
    ct[i] = sumCT;
    tat[i] = (ct[i] - at[i]);
    wt[i] = (tat[i] - bt[i]);
    rt[i] = wt[i];

    sumTAT += tat[i];
    sumWT += wt[i];
    sumRT += rt[i];
}

printf("\n Process\t AT\t BT\t CT\t TAT\t WT\t RT");
for (int i = 0; i < n; i++) {
    printf("\n P%d\t %d\t %d\t %d\t %d\t %d\t %d",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}

printf("\n\n Average Turnaround Time: %.2f", sumTAT/n);
printf("\n Average Waiting Time: %.2f", sumWT/n);
printf("\n Average Response Time: %.2f\n", sumRT/n);
}

```

• Output:-

Enter the number of processes: 4

Enter the arrival time and burst time for each process:

Arrival time for P1: 0

Burst time for P1: 7

Arrival time for P2: 8

Burst time for P2: 3

Arrival time for P3: 3

~~Burst~~ time for P3: 4

Arrival time for P4: 5

Burst time for P4: 6

Process	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P3	3	4	11	8	4	4
P4	5	6	17	12	6	6
P2	8	3	20	12	9	9

Average Turnaround Time: 9.75

Average Waiting Time: 4.75

Average Response Time: 4.75

• SJF:-

```
#include <stdio.h>
```

```
void SJF_NonPreemptive (int id[], int at[], int bt[], int n) {
    int ct[n], tat[n], wt[n], start[n];
```

```
    float sumTAT=0, sumWT=0, sumRT=0;
```

```
    int time=0, completed=0;
```

```
    for (int i=0; i<n; i++) {
```

```
        ct[i] = tat[i] = wt[i] = start[i] = -1;
```

```
    }
```

```
    while (completed < n) {
```

```
        int idx = -1, minBT = 1e9;
```

```
        for (int i=0; i<n; i++) {
```

```
            if (at[i] <= time && ct[i] == -1 && bt[i] < minBT) {
```

```
                minBT = bt[i];
```

```
                idx = i;
```

```
            }
```

```
        }
```

```
        if (idx == -1) {
```

```
            time ++;
```

```
        } else {
```

```
            start[idx] = time;
```

```
            time += bt[idx];
```

```
            ct[idx] = time;
```

```
            tat[idx] = ct[idx] - at[idx];
```

```
            wt[idx] = tat[idx] - bt[idx];
```

```
            int response = ct[idx] - at[idx];
```

9

```

sumTAT += tat[idx];
sumWT += wt[idx];
sumRT += response;
completed++;

```

```

}

printf("\n --- Non-preemptive SJF ---");
printf("\n Process\t AT\t BT\t CT\t TAT\t WT\t RT");
for (int i = 0; i < n; i++) {
    printf("\n P%d\t %d\t %d\t %d\t %d\t %d\t %d\t %d",
           id[i], at[i], bt[i], ct[i], tat[i], wt[i],
           start[i] - at[i]);
}

printf("\n\n Average TAT: %.2f", sumTAT/n);
printf("\n Average WT: %.2f", sumWT/n);
printf("\n Average RT: %.2f\n", sumRT/n);

```

```

}

void SJF_preemptive(int id[], int at[], int bt[], int n) {
    int rt[n], ct[n], tat[n], wt[n], start[n];
    float sumTAT = 0, sumWT = 0, sumRT = 0;
    int time = 0, completed = 0;

    for (int i = 0; i < n; i++) {
        rt[i] = bt[i];
        ct[i] = tat[i] = wt[i] = -1;
        start[i] = -1;
    }

    while (completed < n) {
        int idx = -1, minRT = 1e9;
        for (int i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0 && rt[i] < minRT) {
                minRT = rt[i];
                idx = i;
            }
        }

        if (idx == -1) {
            time++;
        } else {
            rt[idx]--;
            ct[idx]++;
            tat[idx]++;
            wt[idx]++;
            start[idx] = time;
            time++;
            completed++;
        }
    }
}

```



```

else {
    if (start[id * J] == -1) start[id * J] = time;
    nt[id * J]--;
    time++;
    if (nt[id * J] == 0) {
        ct[id * J] = time;
        tat[id * J] = ct[id * J] - at[id * J];
        wt[id * J] = tat[id * J] - bt[id * J];
        int response = start[id * J] - at[id * J];
        sumTAT += tat[id * J];
        sumWT += wt[id * J];
        sumRT += response;
        completed++;
    }
}

printf("\n --- Preemptive SJF ---");
printf("In process\tAT\tBT\tCT\tTAT\tWT\tRT");
for (int i = 0; i < n; i++) {
    printf("\n P%d\t%d\t%d\t%d\t%d\t%d\t%d",
        id[i], at[i], bt[i], ct[i], tat[i], wt[i], start[i] - at[i]);
}

printf("\n\n Average TAT: %.2f", sumTAT/n);
printf("\n Average WT: %.2f", sumWT/n);
printf("\n Average RT: %.2f\n", sumRT/n);
}

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int id[n], at[n], bt[n];
    for (int i = 0; i < n; i++) {
        id[i] = i + 1;
        printf("\n Arrival time for P%d: ", i + 1);
        scanf("%d", &at[i]);
        printf("\n Burst time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    SJF_Nonpreemptive(id, at, bt, n);
    SJF_Preemptive(id, at, bt, n);
}

```

Output:-

11

Enter number of processes: 4

Arrival time for P1: 0

Burst time for P1: 7

Arrival time for P2: 8

Burst time for P2: 3

Arrival time for P3: 3

Burst time for P3: 4

Arrival time for P4: 5

Burst time for P4: 6

--- Non-preemptive SJF ---

Process	AT	BT	CT	TAT	WT	RT
---------	----	----	----	-----	----	----

P1	0	7	7	7	0	0
----	---	---	---	---	---	---

P2	8	3	14	6	3	3
----	---	---	----	---	---	---

P3	3	4	11	8	4	4
----	---	---	----	---	---	---

P4	5	6	20	15	9	9
----	---	---	----	----	---	---

Average TAT: 9.00

Average WT: 4.00

Average RT: 4.00

--- Preemptive SJF ---

Process	AT	BT	CT	TAT	WT	RT
---------	----	----	----	-----	----	----

P1	0	7	7	7	0	0
----	---	---	---	---	---	---

P2	8	3	11	3	7	4
----	---	---	----	---	---	---

P3	3	4	14	11	7	4
----	---	---	----	----	---	---

P4	5	6	20	15	9	9
----	---	---	----	----	---	---

Average TAT: 9.00

Average WT: 4.00

Average RT: 3.25

Comparison b/w FCFS & SJF :-

• In Case of FCFS :-

- Average Turnaround Time : 9.75
- Average Response Time : 4.75
- Average Waiting Time : 4.75

• In Case of SJF (non-preemptive) :-

- Average ~~Turnaround~~ Turnaround Time : 9.00
- Average Waiting Time : 4.00
- Average Response Time : 4.00

→ We can see that Turnaround Time, Waiting Time and Response time all of them are less in ~~case~~ case of SJF (non-preemptive) compared to FCFS.

• In Case of SJF (preemptive) :-

- Average Turnaround Time : 9.00
- Average Waiting Time : 4.00
- Average Response Time : 3.25

→ We have found earlier that SJF (non-preemptive) is better than FCFS.

→ Now, SJF (preemptive) further reduce the response time than SJF (non-preemptive), Rest Turnaround Time, Waiting Time are same.

Hence,

We can conclude that SJF (preemptive) algorithm is best for our inputs that we have provided.

optimal



Priority Scheduling (non-preemptive)

#include <stdio.h>

int main(){

int n;

printf("Enter the number of processes: ");

scanf("%d", &n);

int at[n], bt[n], pr[n], ct[n], tat[n], wt[n], pid[n];

int visited[n];

int time=0, completed=0;

for(int i=0; i<n; i++){

pid[i] = i+1;

visited[i] = 0;

printf("In Arrival Time for P%d: ", i+1);

scanf("%d", &at[i]);

printf("Burst Time for P%d: ", i+1);

scanf("%d", &bt[i]);

printf("Priority for P%d: ", i+1);

scanf("%d", &pr[i]);

}

while(completed < n){

int index = -1;

int highest = 9999;

for(int i=0; i<n; i++){

if(at[i] <= time & visited[i] == 0){

if(pr[i] < highest){

highest = pr[i];

index = i;

}

}

}

```

if (index != -1) {
    time += bt[index];
    ct[index] = time;
    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];
    visited[index] = 1;
    completed++;
}
else {
    time++;
}
}

printf("In Process \t A\t BT \t PR \t CT \t TAT \t WT \n");
for (int i=0; i<n; i++) {
    printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d \n",
        pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}
return 0;
}

```

• Output:-

Enter number of processes: 5

Arrival time for P1: 0

Burst time for P1: 3

Priority for P1: 5

Arrival time for P2: 2

Burst time for P2: 2

Priority for P2: 3

Arrival time for P3: 3

Burst time for P3: 5

Priority for P3: 2

Arrival time for P4: 4

Burst time for P4: 4

Priority for P4: 4

Arrival time for P5: 6
 Burst time for P5: 1
 Priority for P5: 1

Process	AT	BT	PR	CT	TAT	WT	RT
P1	0	3	5	3	3	0	0
P2	2	2	3	11	9	7	7
P3	3	5	2	8	5	0	0
P4	4	4	4	15	11	7	7
P5	6	1	1	9	3	2	2

• Priority Scheduling (Preemptive)

Function for Gantt Chart

```

void ganttChart (int order[], int time[], int k) {
    printf("\n Gantt Chart: \n!");
    for (int i=0; i<k; i++)
        printf(" P%d |", order[i]);
    printf("\n %d", time[0]);
    for (int i=1; i<=k; i++)
        printf(" %d", time[i]);
    printf("\n");
}
  
```

• Output

Gantt Chart:

1 P1 | P3 | P5 | P2 | P4 |
 0 3 8 9 11 15

Average TAT: 6.20

Average WT: 3.20

Average RT: 3.20

• Priority (Preemptive) :-

```
#include <stdio.h>
```

```
void ganttChart(int order[], int time[], int k) {  
    printf("\n Gantt Chart: \n");  
    for (int i=0; i<k; i++) {  
        printf("P %d |", order[i]);  
    }  
    printf("\n %d", time[0]);  
    for (int i=1; i<=k; i++) {  
        printf(" %d", time[i]);  
    }  
    printf("\n");  
}
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], pr[n], rt[n];
```

```
    int ct[n], tat[n], wt[n], resp[n], pid[n];
```

```
    int completed = 0, time = 0;
```

```
    int order[1000];
```

```
    int timeLine[1001];
```

```
    int k=0;
```

```
    float avgTAT=0, avgWT=0, avgRT=0;
```

```
    for (int i=0; i<n; i++) {
```

```
        pid[i] = i+1;
```

```
        printf("\n Arrival Time for P%d: ", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf(" Burst Time for P%d: ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
        printf(" Priority for P%d: ", i+1);
```

```
        scanf("%d", &pr[i]);
```

```
        rt[i] = bt[i];
```

```
        resp[i] = -1;
```

```
    }
```

```
    timeLine[0] = 0;
```

```
while (completed < n) {
```

```
    int index = -1;
```

```
    int highest = 9999;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (at[i] <= time && pt[i] > 0) {
```

```
            if (pn[i] < highest) {
```

```
                highest = pn[i];
```

```
                index = i;
```

```
            }
```

```
            else if (pn[i] == highest) {
```

```
                if (at[i] < at[index])
```

```
                    index = i;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
if (index != -1) {
```

```
    if (resp[index] == -1)
```

```
        resp[index] = time;
```

```
    order[k] = pid[index];
```

```
    k++;
```

```
    time++;
```

```
    pt[index]--;
```

```
    timeLine[k] = time;
```

```
    if (pt[index] == 0) {
```

```
        ct[index] = time;
```

```
        tat[index] = ct[index] - at[index];
```

```
        wt[index] = tat[index] - bt[index];
```

```
        avgTAT += tat[index];
```

```
        avgWT += wt[index];
```

```
        avgRT += resp[index] - at[index];
```

```
        completed++;
```

```
    }
```

```
}
```

```

else {
    order[k] = 0;
    k++;
    time++;
    timeLine[k] = time;
}
}

printf("\n Process \t AT \t BT \t PR \t CT \t TAT \t WT \t RT\n");
for(int i=0; i<n; i++){
    printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d \t %d\n",
        pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i], resp[i]-at[i]);
}

ganntChart(order, timeLine, k);

printf("In Average TAT: %.2f\n", avgTAT/n);
printf("Average WT: %.2f\n", avgWT/n);
printf("Average RT: %.2f\n", avgRT/n);

return 0;
}

```

• Output:-

Enter the number of processes: 5

Arrival time for P1: 0

Burst time for P1: 3

Priority for P1: 5

Arrival time for P2: 2

Burst time for P2: 2

Priority for P2: 3

Arrival time for P3: 3

Burst time for P3: 5

Priority for P3: 2

Arrival time for P4: 4

Burst time for P4: 4

Priority for P4: 4

Arrival time for P5: 6

Burst time for P5: 1

Priority for P5: 1

Process	AT	BT	PR	CT	TAT	WT	RT
P1	0	3	5	15	15	12	0
P2	2	2	3	10	8	6	0
P3	3	5	2	9	6	1	0
P4	4	4	4	14	10	6	0
P5	6	1	1	7	1	6	0

Gantt Chart:

| P1 | P1 | P2 | P3 | P3 | P3 | P5 | P3 | P3 | P2 | P4 | P4 | P4 | P4 |
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

Average TAT: 8.00

Average WT: 5.00

Average RT: 1.20

SK

10/3

Round Robin

#include <stdio.h>

#define MAX 100

int main() {

int n, tq;

printf("Enter number of processes: ");

scanf("%d", &n);

int at[MAX], bt[MAX], rt[MAX], pid[MAX];

int ct[MAX], wt[MAX], tat[MAX], rt_time[MAX];

int first_exec[MAX];

for(int i=0; i<n; i++){

pid[i] = i+1;

printf("Enter AT and BT for P%d: ", i+1);

scanf("%d %d", &at[i], &bt[i]);

rt[i] = bt[i];

first_exec[i] = -1;

}

printf("\n Enter Time Quantum: ");

scanf("%d", &tq);

for(int i=0; i<n-1; i++){

for(int j=i+1; j<n; j++){

if(at[i] > at[j]){

int temp;

temp = at[i]; at[i] = at[j]; at[j] = temp;

temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;

temp = rt[i]; rt[i] = rt[j]; rt[j] = temp;

temp = pid[i]; pid[i] = pid[j]; pid[j] = temp;

}

}

}

int time = at[0], completed = 0;

int q[MAX], front = 0, rear = 0;

int visited[MAX] = {0};

int g_p[MAX], g_t[MAX], g = 0;

```
q[rear++] = 0;
```

```
visited[0] = 1;
```

```
while (completed < n) {
```

```
    int i = q[front++];
```

```
    if (first_exec[i] == -1)
```

```
        first_exec[i] = time;
```

```
    g_p[g] = pid[i];
```

```
    int exec = (nt[i] > tq) ? tq : nt[i];
```

```
    time += exec;
```

```
    nt[i] -= exec;
```

```
    g_t[g] = time;
```

```
    g++;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (nt[j] <= time && !visited[j]) {
```

```
            q[rear++] = j;
```

```
            visited[j] = 1;
```

```
        }
```

```
    }
```

```
    if (nt[i] > 0) {
```

```
        q[rear++] = i;
```

```
    } else {
```

```
        nt[i] = time;
```

```
        completed;
```

```
    }
```

```
    if (front == rear) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (!visited[j]) {
```

```
                time = nt[j];
```

```
                q[rear++] = j;
```

```
                visited[j] = 1;
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```

printf("\n Gantt Chart: |n|");
for (int i=0; i<g; i++){
    printf(" P%d |", g-p[i]);
}

printf("\n %d", at[0]);
for (int i=0; i<g; i++){
    printf(" %d", g-t[i]);
}

float total_wt=0, total_tat=0, total_rt=0;
printf("\n\n Process\tAT\tBT\tCT\tWT\tTAT\tRT\n");

for (int i=0; i<n; i++){
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    rt_time[i] = first_exec[i] - at[i];
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], ct[i], wt[i], tat[i], rt_time[i]);

    total_wt += wt[i];
    total_tat += tat[i];
    total_rt += rt_time[i];
}

printf("\n Average WT = %.2f", total_wt/n);
printf("\n Average TAT = %.2f", total_tat/n);
printf("\n Average RT = %.2f", total_rt/n);

return 0;
}

```

• Output:-

Enter number of processes: 6

Enter AT and BT for P1: 5 5

Enter AT and BT for P2: 4 6

Enter AT and BT for P3: 3 7

Enter AT and BT for P4: 1 9

Enter AT and BT for P5: 2 2

Enter AT and BT for P6: 6 3

Enter Time Quantum: 4

Grant Chart:-

1 P4 | P5 | P3 | P2 | P1 | P4 | P6 | P3 | P2 | P1 | P4 |

Process	AT	BT	CT	WT	TAT	RT
P4	1	9	33	23	32	0
P5	2	2	7	3	5	3
P3	3	7	29	19	26	4
P2	4	6	31	21	27	7
P1	5	5	32	22	27	10
P6	6	3	26	17	20	17

Average WT = 17.50

Average TAT = 22.83

Average RT = 6.83

For Time Quantum = 5

Average WT = 16.50

Average TAT = 21.83

Average RT = 8.00

For Time Quantum = 6

Average WT = 16.17

Average TAT = 21.50

Average RT = 8.67

For Time Quantum = 7

Average WT = 14.83

Average TAT = 20.17

Average RT = 11.00

→ As we can see, as the Time Quantum is increasing average wait time and average Turn around Time are reducing while average response time is increasing.

• Multi Level Queue :-

#include <stdio.h>

#include <string.h>

#define MAX 100

typedef struct {

int id, at, bt, rt, ct, wt, tat;

int type;

} Process;

Process p[MAX];

int gantt_pid[MAX];

int gantt_time[MAX];

int g_index = 0;

void sortByArrival (int n) {

for (int i = 0; i < n-1; i++) {

for (int j = i+1; j < n; j++) {

if (p[i].at > p[j].at) {

Process temp = p[i];

p[i] = p[j];

p[j] = temp;

}

}

}

}

void sortByID (int n) {

for (int i = 0; i < n-1; i++) {

for (int j = i+1; j < n; j++) {

if (p[i].id > p[j].id) {

Process temp = p[i];

p[i] = p[j];

p[j] = temp;

}

}

}

}

```

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i=0; i<n; i++) {
        printf("Process %d\n", i);
        p[i].id = i;
        printf("Enter arrival time: ");
        scanf("%d", &p[i].at);
        printf("Enter burst time: ");
        scanf("%d", &p[i].bt);
        printf("Enter type (0=System, 1=User): ");
        scanf("%d", &p[i].type);
        p[i].rt = p[i].bt;
    }

    sortByArrival(n);

    int time = 0, completed = 0;
    int current = -1;

    while (completed < n) {
        int highest = -1;
        for(int i=0; i<n; i++) {
            if(p[i].at <= time && p[i].rt > 0) {
                if(highest == -1)
                    highest = i;
                else {
                    if(p[i].type < p[highest].type)
                        highest = i;
                    else if (p[i].type == p[highest].type &&
                        p[i].at < p[highest].at)
                        highest = i;
                }
            }
        }

        if (highest == -1) {
            time++;
            continue;
        }
    }
}

```

```
if (g_index == 0 || gantt_pid[g_index-1] !=
```

```
    p[highest].id){
```

```
    gantt_pid[g_index] = p[highest].id;
```

```
    gantt_time[g_index] = time;
```

```
    g_index++;
```

```
}
```

```
p[highest].rt--;
```

```
time++;
```

```
if (p[highest].rt == 0){
```

```
    p[highest].ct = time;
```

```
    completed++;
```

```
}
```

```
}
```

```
gantt_time[g_index] = time;
```

```
float total_wt = 0, total_tat = 0;
```

```
for (int i = 0; i < n; i++){
```

```
    p[i].tat = p[i].ct - p[i].rt;
```

```
    p[i].wt = p[i].tat - p[i].bt;
```

```
    total_wt += p[i].wt;
```

```
    total_tat += p[i].tat;
```

```
}
```

```
sortByID(n);
```

```
printf("\n ID \t Type \t AT \t BT \t CT \t WT \t TAT \n");
```

```
for (int i = 0; i < n; i++){
```

```
    printf("%d \t %d \t %d \t %d \t %d \t %d \t %d \n",
```

```
        p[i].id;
```

```
        (p[i].type == 0) ? "System" : "User",
```

```
        p[i].at;
```

```
        p[i].bt;
```

```
        p[i].ct;
```

```
        p[i].wt;
```

```
        p[i].tat);
```

```
}
```

```

printf("\n Average Waiting Time : %.2f", total-wt/n);
printf("\n Average Turnaround Time : %.2f\n", total-tat/n);

printf("\n Gantt Chart : \n");
printf("|");
for (int i=0; i < g-index; i++) {
    printf("P%d |", gantt-pid[i]);
}
printf("\n %d", gantt-time[0]);
for (int i=1; i <= g-index; i++) {
    printf("    %d", gantt-time[i]);
}
printf("\n");
return 0;
}

```

• Output :-

Enter number of processes : 4

Process 0

Enter arrival time : 0

Enter burst time : 5

Enter type (0 = System, 1 = User) : 0

Process 1

Enter arrival time : 2

Enter burst time : 3

Enter type (0 = System, 1 = User) : 1

Process 2

Enter arrival time : 1

Enter burst time : 4

Enter type (0 = System, 1 = User) : 0

Process 3

Enter arrival time : 3

Enter burst time : 2

Enter type (0 = System, 1 = User) : 1

ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
1	User	2	3	12	7	10
2	System	1	4	9	4	8
3	User	3	2	14	9	11

Average Waiting Time : 5.00

Average Turnaround Time : 8.50

Gantt Chart:

| P0 | 1 | P2 | 1 | P1 | 1 | P3 | 1 |
 0 5 9 12 14


 24/3

```
#include <stdio.h>
#include <math.h>
```

```
struct Process {
    int id;
    int burst;
    int deadline;
    int period;
    int completion;
    int waiting;
    int turnaround;
};
```

```
void sortProcesses(struct Process p[], int n, char key) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            int a, b;
            if (key == 'd') {
                a = p[i].deadline; b = p[j].deadline;
            }
            else if (key == 'p') {
                a = p[i].period; b = p[j].period;
            }
            else {
                a = p[i].id; b = p[j].id;
            }
            if (a > b) {
                struct Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
```

```

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct process p[n];
    printf("\n Enter process details: \n");
    for (int i=0; i<n; i++) {
        p[i].id = i;
        printf("\n Process %d: \n", i);
        printf("Burst time: ");
        scanf("%d", &p[i].burst);
        printf("Deadline (for EDF): ");
        scanf("%d", &p[i].deadline);
        printf("Period (for RMS): ");
        scanf("%d", &p[i].period);
    }

```

```

double utilization = 0.0;
for (int i=0; i<n; i++) {
    utilization += (double) p[i].burst / p[i].period;
}

```

```

printf("\n --- Earliest Deadline First (EDF) Scheduling --- \n");
printf("CPU Utilization: %.2f \n", utilization);
if (utilization <= 1.0)
    printf("Schedulable (utilization <= 1) \n");
else
    printf("Not Schedulable \n");

```

```

sortProcesses(p, n, 'd');

```

```

int time = 0;

```

```

for (int i=0; i<n; i++) {

```

```

    time += p[i].burst;

```

```

    p[i].completion = time;

```

```

    p[i].turnaround = p[i].completion;

```

```

    p[i].waiting = p[i].turnaround - p[i].burst;

```

```

}

```

```

sortProcesses(p, n, 'i');

```

```

printf("ID BF Deadline CT WT TAT\n");
for (int i=0; i<n; i++) {
    printf("%d %d %d %d %d %d\n",
        p[i].id, p[i].burst, p[i].deadline,
        p[i].completion, p[i].waiting, p[i].turnaround);
}

```

```

utilization = 0.0

```

```

for (int i=0; i<n; i++) {
    utilization += (double) p[i].burst / p[i].period;
}

```

```

double rm_bound = n * (pow(2.0, 1.0/n) - 1);

```

```

printf("--- Rate Monotonic Scheduling (RMS) ---\n");
printf("CPU Utilization: %.2f\n", utilization);
printf("RM bound: %.4f\n", rm_bound);
if (utilization <= rm_bound)
    printf("Schedulable\n");

```

```

else
    printf("Not schedulable\n");

```

```

sortProcesses(p, n, 'r');

```

```

time = 0;

```

```

for (int i=0; i<n; i++) {
    time += p[i].burst;
    p[i].completion = time;
    p[i].turnaround = p[i].completion;
    p[i].waiting = p[i].turnaround - p[i].burst;
}

```

```

sortProcesses(p, n, 't');

```

```

printf("ID BF Period CT WT TAT\n");

```

```

for (int i=0; i<n; i++) {
    printf("%d %d %d %d %d %d\n",
        p[i].id, p[i].burst, p[i].period,
        p[i].completion, p[i].waiting, p[i].turnaround);
}

```

```

}

```


• Output:-

Enter number of processes : 3

Enter process details:

Process 0:

Burst Time: 3

Deadline (for EDF): 7

Period (for RMS): 20

Process 1:

Burst Time: 2

Deadline (for EDF): 4

Period (for RMS): 5

Process 2

Burst Time: 2

Deadline (for EDF): 8

Period (for RMS): 10

--- Earliest Deadline First (EDF) Scheduling ---

CPU Utilization: 0.75

Schedulable (Utilization ≤ 1)

ID	BF	Deadline	CT	WT	TAT
0	3	7	5	2	5
1	2	4	2	0	2
2	2	8	7	5	7

--- Rate Monotonic Scheduling (RMS) ---

CPU Utilization: 0.75

RM Bound: 0.7798

Schedulable (Utilization $\leq$ RM Bound)

ID	BF	Deadline	CT	WT	TAT
0	3	20	7	4	7
1	2	5	2	0	2
2	2	10	4	2	4

SK

3/13

Q → The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0 = 1, S_1 = 0, S_2 = 0$. Find out how many times Process P0 will print "o". Assume the order of execution as P0, P1, P2, P0, P1.

Process P0: while (true) { wait(S0); printf("o"); signal(S1); signal(S2); }	Process P1: wait(S1); signal(S0);	Process P2: wait(S2); signal(S0);
--	--	--

Solⁿ:

Initial	$S_0 = 1$	$S_1 = 0$	$S_2 = 0$	Remark
P0	0	1	1	o
P1	1	0	1	—
P2	1	0	0	—
P0	0	1	1	oo
P1	1	0	1	—

→ Process P0 will print "o" two times

Q → Consider two processes A and B. Two semaphore variable S and T are used to synchronize the processes. S is initialized to 0 and T is initialized to 1. Processes are scheduled in the following order: A B A B B A A B A. What will be printed on the screen?

Process A: while (1) { wait(S); print("P"); print("P"); signal(T); }	Process B: while (1) { wait(T); print("I"); print("I"); signal(S); }
---	---

Solⁿ:-

Initial	S = 0	T = 1	Remark
A	0	1	Blocked
B	1	0	II
A	0	1	PP
B	1	0	II
B	1	0	Blocked
A	0	1	PP
A	0	1	Blocked
B	1	0	II
A	0	1	PP

∴ Final Ans:- IIPP II PP II PP

PPPP P

• • Bounded Buffering

Sequence → P C P C P C P P C

Buffer-Size = 5

in = 0, out = 0

empty = 5, full = 0, mutex = 1

Step	Operation	Buffer state	in	out	empty	full	mutex
1	Produced: 0	[0, -, -, -, -]	1	0	4	1	1
2	Consumed: 0	[-, -, -, -, -]	1	1	5	0	1
3	Produced: 1	[-, 1, -, -, -]	2	1	4	1	1
4	Consumed: 1	[-, -, -, -, -]	2	2	5	0	1
5	Produced: 2	[-, -, 2, -, -]	3	2	4	1	1
6	Consumed: 2	[-, -, -, -, -]	3	3	5	0	1
7	Produced: 3	[-, -, -, 3, -]	4	3	4	1	1
8	Produced: 4	[-, -, -, 3, 4]	4	3	3	2	1
9	Consumed: 3	[-, -, -, 4, -]	4	4	4	1	1

Dining Philosophers:-

CS = [1, 1, 1, 1, 1]

P0 is thinking.

P1 is thinking.

P2 is thinking.

P3 is thinking.

P4 is thinking.

P4 picked left fork 4. $\rightarrow$ CS = [1, 1, 1, 1, 0]

P2 picked left fork 1 $\rightarrow$ CS = [1, 0, 1, 1, 0]

P1 picked right fork 2 $\rightarrow$ CS = [1, 0, 0, 1, 0]

P1 is eating.

P3 picked left fork 3 $\rightarrow$ CS = [1, 0, 0, 0, 0]

P0 picked left fork 0 $\rightarrow$ CS = [0, 0, 0, 0, 0]

P1 put down forks 1 and 2 $\rightarrow$ CS = [0, 1, 1, 0, 0]

P1 is thinking.

P2 picked left fork 2 $\rightarrow$ CS = [0, 1, 0, 0, 0]

P0 picked right fork 1 $\rightarrow$ CS = [0, 0, 0, 0, 0]

P0 is eating.

P0 put down forks 0 and 1 $\rightarrow$ CS = [1, 1, 0, 0, 0]

P4 picked right fork 0. $\rightarrow$ CS = [0, 1, 0, 0, 0]

P4 is eating.

P0 is thinking.

8/12
2/14

• Bounded Buffer :-

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <unistd.h>
```

```
#define BUFFER_SIZE 5
#define MAX_ITEMS 15
```

```
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
```

```
sem_t empty;
```

```
sem_t full;
```

```
pthread_mutex_t mutex;
```

```
void *producer(void *arg){
```

```
    for (int i=0; i<MAX_ITEMS; i++){
```

```
        int item = i;
```

```
        sem_wait(&empty);
```

```
        pthread_mutex_lock(&mutex);
```

```
        buffer[in] = item;
```

```
        printf("Producer: %d at buffer [%d]\n", item, in);
```

```
        in = (in+1) % BUFFER_SIZE;
```

```
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&full);
```

```
        usleep(100000);
```

```
    }
```

```
    return NULL;
```

```
}
```

```
void *consumer(void *arg){
```

```
    for (int i=0; i<MAX_ITEMS; i++){
```

```
        sem_wait(&full);
```

```
        pthread_mutex_lock(&mutex);
```

```
        int item = buffer[out];
```

```
        printf("Consumer: %d from buffer [%d]\n", item, out);
```

```
        out = (out+1) % BUFFER_SIZE;
```

```
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&empty);
```

```
        usleep(150000);
```

```
    }
```

```
    return NULL;
```

```

int main() {
    pthread_t prod, cons;
    sem_t empty, full;
    pthread_mutex_t mutex;
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}

```

• Dining Philosopher

```

#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <unistd.h>
#include <csignal.h>

#define N 5

sem_t forks[N];
sem_t room;

void think(int i) {
    printf("Philosopher %d is thinking.\n", i);
    usleep(100000);
}

void eat(int i) {
    printf("Philosopher %d is eating.\n", i);
    usleep(150000);
}

void *philosopher(void *arg) {
    int i = *(int *)arg;
    while(1) {
        think(i);
        sem_wait(&room);
        sem_wait(&forks[i]);
        printf("Philosopher %d picked up left fork %d.\n", i, i);
    }
}

```

```

sem_wait(&forks[(i+1)%N]);
printf("Philosopher %d picked up right fork %d.\n", i, (i+1)%N);
eat(i);
sem_post(&forks[i]);
sem_post(&forks[(i+1)%N]);
printf("Philosopher %d put down forks %d and %d.\n",
      i, i, (i+1)%N);
sem_post(&room);
}
return NULL;
}

int main() {
pthread_t phil[N];
int ids[N];
sem_init(&room, 0, N-1);
for (int i=0; i<N; i++) {
sem_init(&forks[i], 0, 1);
}
for (int i=0; i<N; i++) {
ids[i] = i;
pthread_create(&phil[i], NULL, philosopher, &ids[i]);
}
for (int i=0; i<N; i++) {
pthread_join(phil[i], NULL);
}
return 0;
}

```


28/04/21
Q → Consider the following snapshot of a system A, B, C with values
LAB 7 10, 5, 7.

Process	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	3	3	2	7	4	3
P ₁	2	0	0	3	2	2				1	2	2
P ₂	3	0	2	9	0	2				6	0	0
P ₃	2	1	1	2	2	2				0	1	1
P ₄	0	0	2	4	3	3				4	3	1
	7	2	5									

→ Need = Max - Allocation

Safe Sequence :- {P₁, P₃, P₄, P₀, P₂}

P₀
Finish[0] = false
Need ≤ available
743 < 332

P₁
Finish[1] = false
122 ≤ 332
Work = Work + Allocation
= 332 + 200 = 532
Finish[1] = true

P₂
Finish[2] = false
600 < 532

P₃
Finish[3] = false
011 ≤ 532
Work = Work + Allocation
= 532 + 302 = 834
Finish[3] = true

P₄
Finish[4] = false
431 ≤ 743
Work = Work + Allocation
= 743 + 002 = 745
Finish[4] = true

P₀
Finish[0] = false
743 ≤ 745
Work = Work + Allocation
= 745 + 010 = 755
Finish[0] = true

P₂
Finish[2] = false
600 ≤ 755
Work = Work + Allocation
= 755 + 302 = 1057
Finish[2] = true

{P₁, P₃, P₄, P₀, P₂}

Q → Consider the following snapshot of a system:

	Allocation	Max	Available	Need
	A B C D	A B C D	A B C D	A B C D
P0	0 0 1 2	0 0 1 2	1 5 2 0	0 0 0 0
P1	1 1 0 0	1 7 5 0	5 0 2	0 7 5 0
P2	1 1 8 5	2 3 5 6	1 1 5	1 0 0 2
P3	0 6 3 2	0 6 5 2	5 0 0	0 0 2 0
P4	0 0 1 4	0 6 5 6	2 5 5	0 6 4 2

Total:-

$$\begin{array}{r} 2 \ 9 \ 10 \ 12 \\ 1 \ 5 \ 2 \ 0 \\ \hline 3 \ 14 \ 12 \ 12 \end{array}$$

P0
 Finish[0] = false
 Need ≤ available
 $0 \ 0 \ 0 \ 0 \leq 1 \ 5 \ 2 \ 0$
 Work = Work + Allocation
 $= 1 \ 5 \ 2 \ 0 + 0 \ 0 \ 1 \ 2$
 $= 1 \ 5 \ 3 \ 2$
 Finish[0] = true

P1
 Finish[1] = false
 Need ≤ available
 $0 \ 7 \ 5 \ 0 \not\leq 1 \ 5 \ 3 \ 2$

P2
 Finish[2] = false
 $1 \ 0 \ 0 \ 2 \not\leq 1 \ 5 \ 3 \ 2$
 Work = Work + Allocation
 $= 1 \ 5 \ 3 \ 2 + 1 \ 1 \ 8 \ 5$
 $= 2 \ 6 \ 11 \ 7$
 Finish[2] = true

P3
 Finish[3] = false
 $0 \ 0 \ 2 \ 0 \leq 2 \ 6 \ 11 \ 7$
 Work = Work + Allocation
 $= 2 \ 6 \ 11 \ 7 + 0 \ 6 \ 3 \ 2$
 $= 2 \ 12 \ 14 \ 9$
 Finish[3] = true

P4
 Finish[4] = false
 $0 \ 6 \ 4 \ 2 \leq 2 \ 12 \ 14 \ 9$
 Work = Work + Allocation
 $= 2 \ 12 \ 14 \ 9 + 0 \ 0 \ 1 \ 4$
 $= 2 \ 12 \ 15 \ 13$
 Finish[4] = true

P1
 Finish[1] = false
 $0 \ 7 \ 5 \ 0 \leq 2 \ 12 \ 15 \ 13$
 Work = Work + Allocation
 $= 2 \ 12 \ 15 \ 13 + 1 \ 0 \ 0 \ 0$
 $= 3 \ 12 \ 15 \ 13$
 Finish[1] = true

Final
 Safe :- {P0, P2, P3, P4, P1}
 Sequence

• Code:-

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main() {
```

```
    int n, m;
```

```
    int allocation[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
```

```
    int available[MAX], work[MAX];
```

```
    int finish[MAX] = {0};
```

```
    int safeSequence[MAX];
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter Allocation Matrix : \n");
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<m; j++) {
```

```
            scanf("%d", &allocation[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Enter Maximum Demand Matrix : \n");
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<m; j++) {
```

```
            scanf("%d", &max[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Enter Available Resources: \n");
```

```
    for (int i=0; i<m; i++) {
```

```
        scanf("%d", &available[i]);
```

```
        work[i] = available[i];
```

```
    }
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<m; j++) {
```

```
            need[i][j] = max[i][j] - allocation[i][j];
```

```
        }
```

```
    }
```

```
    int count = 0;
```

```
while (count < n) {
```

```
    int found = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (finish[i] == 0) {
```

```
            int j;
```

```
            for (j = 0; j < m; j++) {
```

```
                if (need[i][j] > work[j])
```

```
                    break;
```

```
            }
```

```
            if (j == m) {
```

```
                for (int k = 0; k < m; k++) {
```

```
                    work[k] += allocation[i][k];
```

```
                }
```

```
                safeSequence[count++] = i;
```

```
                finish[i] = 1;
```

```
                found = 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    if (found == 0) {
```

```
        printf("System is in an unsafe state (Dead lock possible)");
```

```
        return 0;
```

```
    }
```

```
}
```

```
printf("System is in a Safe State. In Safe Sequence is:");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%d", safeSequence[i]);
```

```
    if (i != n - 1)
```

```
        printf("->");
```

```
}
```

```
printf("\n");
```

```
return 0;
```

```
}
```


• Output:-

Enter number of processes: 5

Enter number of resources: 4

Enter Allocation Matrix:

0 0 1 2

1 0 0 0

1 3 5 4

0 6 3 2

0 0 1 4

Enter Maximum Demand Matrix:

0 0 1 2

1 7 5 0

2 3 5 6

0 6 5 2

0 6 5 6

Enter Available Resources:

1 5 2 0

System is in a Safe State.

Safe sequence is: $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$


28/4

- Banker's Safety Algorithm with need request allocation :-

```
int request[MAX], process;
```

```
// New request
```

```
printf("Enter process number making request: ");
```

```
scanf("%d", &process);
```

```
printf("Enter request for process P%d: \n", process);
```

```
for (int i=0; i<m; i++)
```

```
scanf("%d", &request[i]);
```

```
// Check Request  $\leq$  Need
```

```
for (int i=0; i<m; i++) {
```

```
if (request[i] > need[process][i]) {
```

```
printf("Error: Request exceed process need! \n");
```

```
return 0;
```

```
}
```

```
}
```

```
// Check Request  $\leq$  Available
```

```
for (int i=0; i<m; i++) {
```

```
if (request[i] > available[i]) {
```

```
printf("Resources not available. Process must wait. \n");
```

```
return 0;
```

```
}
```

```
}
```

```
// Pretend Allocation
```

```
for (int i=0; i<m; i++) {
```

```
available[i] -= request[i];
```

```
allocation[process][i] += request[i];
```

```
need[process][i] -= request[i];
```

```
}
```

```
// Initialize work = available
```

```
for (int i=0; i<m; i++)
```

```
work[i] = available[i];
```

```
// main function at the end
```

```
printf("Request granted successfully. \n");
```

• New Output:-

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

Enter Process Number making request: 1

Enter request for ~~making~~ process P1:

1 0 2

System is in SAFE state.

Safe sequence: ~~P1~~ → P3 → P4 → P0 → P2

~~Request granted successfully.~~

• Deadlock Detection 8.

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main() {
```

```
    int n, m;
```

```
    int allocation[MAX][MAX], request[MAX][MAX];
```

```
    int available[MAX], work[MAX];
```

```
    int finish[MAX] = {0};
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resource types: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter Allocation Matrix: \n");
```

```
    for (int i=0; i<n; i++)
```

```
        for (int j=0; j<m; j++)
```

```
            scanf("%d", &allocation[i][j]);
```

```
    printf("Enter Request Matrix: \n");
```

```
    for (int i=0; i<n; i++)
```

```
        for (int j=0; j<m; j++)
```

```
            scanf("%d", &request[i][j]);
```

```
    printf("Enter Available Resources: \n");
```

```
    for (int i=0; i<m; i++) {
```

```
        scanf("%d", &available[i]);
```

```
        work[i] = available[i];
```

```
    }
```

```
    for (int i=0; i<n; i++) {
```

```
        int zero = 1;
```

```
        for (int j=0; j<m; j++) {
```

```
            if (allocation[i][j] != 0) {
```

```
                zero = 0;
```

```
                break;
```

```
            }
```

```
        }
```

```
        if (zero)
```

```
            finish[i] = 1;
```

```
    }
```

```

int found = 1;
while(found) {
    found = 0;
    for (int i=0; i<n; i++) {
        if (finish[i] == 0) {
            int j;
            for (j=0; j<m; j++) {
                if (request[i][j] > work[j])
                    break;
            }
            if (j == m) {
                for (int k=0; k<m; k++)
                    work[k] += allocation[i][k];
                finish[i] = 1;
                found = 1;
            }
        }
    }
}

```

```

int deadlock = 0;
for (int i=0; i<n; i++) {
    if (finish[i] == 0) {
        printf("Process %d is DEADLOCKED\n", i);
        deadlock = 1;
    }
}

```

```

if (!deadlock)
    printf("No deadlock detected, System is Safe.\n");

```

```

return 0;

```

```

}

```


• Output:-

Enter number of processes; 4

Enter number of resource types; 3

Enter Allocation Matrix:

1 0 2

2 1 1

1 0 3

1 2 2

Enter Request Matrix:

0 0 1

1 0 2

0 0 0

3 3 0

Enter Available Resources:

0 0 0

Process P3 is DEADLOCKED.

Process	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	1	0	2	0	0	1	0	0	0
P1	2	1	1	1	0	2			
P2	1	0	3	0	0	0			
P3	1	2	2	3	3	0			

• P0

Request $\leq$ Available

0 0 1 $\leq$ 0 0 0 X

• P1

Request $\leq$ Available

1 0 2 $\leq$ 0 0 0 X

• P2

Request $\leq$ Available

0 0 0 $\leq$ 0 0 0

Work = Work + Allocation

= 0 0 0 + 1 0 3

= 1 0 3

• P3

Request $\leq$ Available

1 3 3 0 $\leq$ 1 0 3 X

• P0

Request $\leq$ Available

0 0 1 $\leq$ 1 0 3 ✓

Work = Work + Allocation

= 1 0 3 + 1 0 2

= 2 0 5

• P1

Request $\leq$ Available

1 0 2 $\leq$ 2 0 5 ✓

Work = Work + Allocation

= 2 0 5 + 2 1 1

= 4 1 6

• P3

Request $\leq$ Available

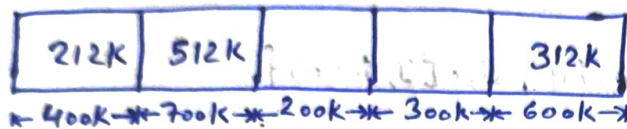
3 3 0 $\leq$ 4 1 6 X

→ P3 is DEADLOCKED

Q → Given a free memory partition 400k, 700k, 200k, 300k & 600k, How would of each of first fit, best fit & worst fit algorithm work to place process of 212k, 512k, 312k, 526k (in order)
 P1 P2 P3 P4

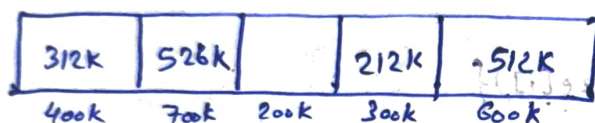
Sol:-

• First fit:-

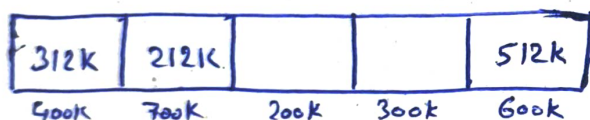


→ 526k cannot be allocated with memory

• Best fit:-



• Worst fit:-



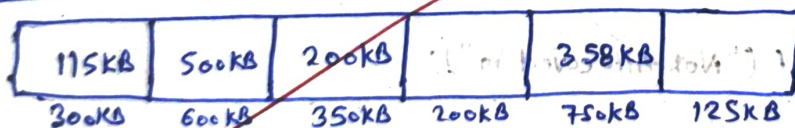
→ 526k cannot be allocated with memory

Q → Memory Partition:- 300KB, 600KB, 350KB, 200KB, 750KB, 125KB (in order)

Process Size:- 115KB, 500KB, 358KB, 200KB, 375KB (in order)

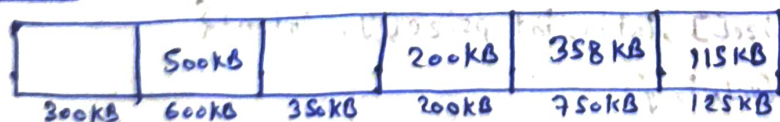
Sol:-

• First fit:-



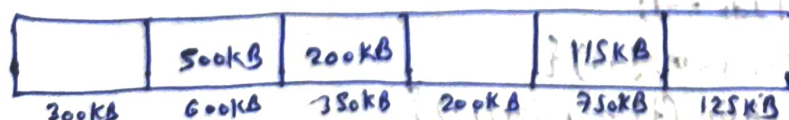
→ 375KB cannot be allocated with memory

• Best fit:-



→ 375KB cannot be allocated with memory

• Worst fit:-



→ Process with size 358KB and 375KB cannot be allocated with memory.

Q → Write a C program to simulate the following contiguous memory allocation techniques.

- First Fit
- Best Fit
- Worst Fit

• Code:-

```
#include <stdio.h>
#define MAX 20

void firstFit(int bsize[], int m, int psize[], int n) {
    int allocation[n], i, j;
    for (i = 0; i < n; i++) allocation[i] = -1;

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            if (bsize[j] >= psize[i]) {
                allocation[i] = j;
                bsize[j] -= psize[i];
                break;
            }
        }
    }

    printf("\n Process No. \t Process Size \t Block no. \n");
    for (i = 0; i < n; i++) {
        printf("%d \t \t %d \t \t", i+1, psize[i]);
        if (allocation[i] != -1)
            printf("%d \n", allocation[i]+1);
        else
            printf("Not Allocated \n");
    }
}

void bestFit(int bsize[], int m, int psize[], int n) {
    int allocation[n], i, j;
    for (i = 0; i < n; i++) allocation[i] = -1;

    for (i = 0; i < n; i++) {
        int bestIdx = -1;
        for (j = 0; j < m; j++) {
            if (bsize[j] >= psize[i]) {
                if (bestIdx == -1 || bsize[bestIdx] > bsize[j])
                    bestIdx = j;
            }
        }
    }
}
```



```

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            bsize[bestIdx] -= psize[i];
        }
    }

    printf("\n Process No. \t Process Size \t Block no. \n");
    for (i=0; i<n; i++) {
        printf("%d \t \t %d \t \t", i+1, psize[i]);
        if (allocation[i] != -1)
            printf("%d \n", allocation[i]+1);
        else
            printf("Not Allocated \n");
    }
}

void worstFit(int bsize[], int m, int psize[], int n) {
    int allocation[n], i, j;
    for (i=0; i<n; i++) allocation[i] = -1;
    for (i=0; i<n; i++) {
        int worstIdx = -1;
        for (j=0; j<m; j++) {
            if (bsize[j] >= psize[i]) {
                if (worstIdx == -1 || bsize[worstIdx] < bsize[j])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            bsize[worstIdx] -= psize[i];
        }
    }

    printf("\n Process No. \t Process Size \t Block no. \n");
    for (i=0; i<n; i++) {
        printf("%d \t \t %d \t \t", i+1, psize[i]);
        if (allocation[i] != -1)
            printf("%d \n", allocation[i]+1);
        else
            printf("Not Allocated \n");
    }
}

```



```

int main() {
    int m, n, i, choice;
    int bsize[MAX], psize[MAX], tempBsize[MAX];
    printf("Enter number of blocks: ");
    scanf("%d", &m);
    printf("Enter size of each block: \n");
    for (i=0; i<m; i++) scanf("%d", &bsize[i]);
    do {
        for (i=0; i<m; i++) tempBsize[i] = bsize[i];
        printf("\n --- Memory Allocation Menu --- ");
        printf("\n 1. First Fit\n 2. Best Fit\n 3. Worst Fit\n 4. Exit\n Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: firstFit(tempBsize, m, psize, n); break;
            case 2: bestFit(tempBsize, m, psize, n); break;
            case 3: worstFit(tempBsize, m, psize, n); break;
            case 4: printf("Exiting -- \n"); break;
            default: printf("Invalid choice! \n");
        }
    } while (choice != 4);
    return 0;
}

```

• Output:-

```

Enter number of blocks: 5
Enter size of each block:
100 500 200 300 600
Enter number of processes: 4
Enter size of each process:
212 417 112 426

```

--- Memory Allocation Menu ---

1. First Fit
2. Best Fit
3. Worst Fit
4. Exit

Enter choice: 1

Process No.	Process Size	Block no.
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

--- Memory Allocation Menu ---

1. First Fit
2. Best Fit
3. Worst Fit
4. Exit

Enter choice: 2

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

--- Memory Allocation Menu ---

1. First Fit
2. Best Fit
3. Worst Fit
4. Exit

Enter choice: 3

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

--- Memory Allocation Menu ---

1. First Fit
2. Best Fit
3. Worst Fit
4. Exit

Enter choice: 4

Exiting...

Signature
12/5

- Q.7) Implement Page replacement algorithms
- i) FIFO
 - ii) LRU
 - iii) Optimal

Sol:-

```
#include <stdio.h>
#define MAX 50
```

```
int search(int frames[], int capacity, int page){
    for(int i=0; i<capacity; i++){
        if(frames[i] == page)
            return i;
    }
    return -1;
}
```

```
void FIFO(int pages[], int n, int capacity){
    int frames[capacity];
    int index = 0, faults = 0;

    for(int i=0; i<capacity; i++)
        frames[i] = -1;

    printf("\n --- FIFO --- \n");

    for(int k=0; k<n; k++){
        int found = search(frames, capacity, pages[k]);
        if(found == -1){
            frames[index] = pages[k];
            index = (index + 1) % capacity;
            faults++;
        }
        printf("Page %d -> ", pages[k]);
        for(int i=0; i<capacity; i++){
            if(frames[i] != -1)
                printf("%d ", frames[i]);
            else
                printf("- ");
        }
        printf("\n");
    }
    printf("Total page faults = %d\n", faults);
}
```

```
void LRU(int pages[], int n, int capacity) {
```

```
    int frames[capacity];
```

```
    int recent[capacity];
```

```
    int faults = 0, time = 0;
```

```
    for (int i = 0; i < capacity; i++) {
```

```
        frames[i] = -1;
```

```
        recent[i] = 0;
```

```
    }
```

```
    printf("\n --- LRU --- \n");
```

```
    for (int k = 0; k < n; k++) {
```

```
        int pos = search(frames, capacity, pages[k]);
```

```
        if (pos != -1) {
```

```
            time++;
```

```
            recent[pos] = time;
```

```
        }
```

```
        else {
```

```
            int lru = 0;
```

```
            for (int i = 1; i < capacity; i++) {
```

```
                if (recent[i] < recent[lru])
```

```
                    lru = i;
```

```
            }
```

```
            time++;
```

```
            frames[lru] = pages[k];
```

```
            recent[lru] = time;
```

```
            faults++;
```

```
        }
```

```
        printf("Page %d -> ", pages[k]);
```

```
        for (int i = 0; i < capacity; i++) {
```

```
            if (frames[i] != -1)
```

```
                printf("%d ", frames[i]);
```

```
            else
```

```
                printf("- ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    printf("Total Page faults = %d \n", faults);
```

```
}
```



```

void OPTIMAL (int pages[], int n, int capacity) {
    int frames[capacity];
    int faults = 0;
    for (int i = 0; i < capacity; i++)
        frames[i] = -1;

    printf("--- OPTIMAL ---\n");
    for (int k = 0; k < n; k++) {
        int found = search (frames, capacity, pages[k]);
        if (found == -1) {
            int replace = -1;
            int farthest = k;
            for (int i = 0; i < capacity; i++) {
                int j;
                for (j = k + 1; j < n; j++) {
                    if (frames[i] == pages[j]) {
                        if (j > farthest) {
                            farthest = j;
                            replace = i;
                        }
                    }
                }
                break;
            }
            if (j == n) {
                replace = i;
                break;
            }
        }
        if (replace == -1)
            replace = 0;
        frames[replace] = pages[k];
        faults++;
    }
    printf("Page %d -> ", pages[k]);
    for (int i = 0; i < capacity; i++) {
        if (frames[i] != -1)
            printf("%d ", frames[i]);
        else
            printf("- ");
    }
    printf("\n");
}
printf("Total Page faults = %d\n", faults);
}

```

```

int main() {
    int pages[MAX];
    int n, capacity;

    printf("Enter number of pages:");
    scanf("%d", &n);

    printf("Enter page reference string: \n");

    for (int i=0; i<n; i++)
        scanf("%d", &pages[i]);

    printf("Enter frame capacity:");
    scanf("%d", &capacity);

    FIFO(pages, n, capacity);
    LRU(pages, n, capacity);
    OPTIMAL(pages, n, capacity);

    return 0;
}

```

• Output:-

Enter number of pages: 20

Enter page reference string:

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Enter frame capacity: 3

--- FIFO ---

Page 7 → 7 - -

Page 0 → 7 0 -

Page 1 → 7 0 1

Page 2 → 2 0 1

Page 0 → 2 0 1

Page 3 → 2 3 1

Page 0 → 2 3 0

Page 4 → 4 3 0

Page 2 → 4 2 0

Page 3 → 4 2 3

Page 0 → 0 2 3

Page 3 → 0 2 3

Page 2 → 0 2 3

Page 1 → 0 1 3

Page 2 → 0 1 2

Page 0 → 0 1 2

Page 1 → 0 1 2

Page 7 $\rightarrow$ 7 1 2
 Page 0 $\rightarrow$ 7 0 2
 Page 1 $\rightarrow$ 7 0 1
 Total Page Faults = 15

--- LRU ---

Page 7 $\rightarrow$ 7 - -
 Page 0 $\rightarrow$ 7 0 -
 Page 1 $\rightarrow$ 7 0 1
 Page 2 $\rightarrow$ 2 0 1
 Page 0 $\rightarrow$ 2 0 1
 Page 3 $\rightarrow$ 2 0 3
 Page 0 $\rightarrow$ 2 0 3
 Page 4 $\rightarrow$ 4 0 3
 Page 2 $\rightarrow$ 4 0 2
 Page 3 $\rightarrow$ 4 3 2
 Page 0 $\rightarrow$ 0 3 2
 Page 3 $\rightarrow$ 0 3 2
 Page 2 $\rightarrow$ 0 3 2
 Page 1 $\rightarrow$ 1 3 2
 Page 2 $\rightarrow$ 1 3 2
 Page 0 $\rightarrow$ 1 0 2
 Page 1 $\rightarrow$ 1 0 2
 Page 7 $\rightarrow$ 1 0 7
 Page 0 $\rightarrow$ 1 0 7
 Page 1 $\rightarrow$ 1 0 7
 Total Page Faults = 12

--- OPTIMAL ---

Page 7 $\rightarrow$ 7 - -
 Page 0 $\rightarrow$ 7 0 -
 Page 1 $\rightarrow$ 7 0 1
 Page 2 $\rightarrow$ 2 0 1
 Page 0 $\rightarrow$ 2 0 1
 Page 3 $\rightarrow$ 2 0 3
 Page 0 $\rightarrow$ 2 0 3
 Page 4 $\rightarrow$ 2 4 3
 Page 2 $\rightarrow$ 2 4 3
 Page 3 $\rightarrow$ 2 4 3
 Page 0 $\rightarrow$ 2 0 3
 Page 3 $\rightarrow$ 2 0 3
 Page 2 $\rightarrow$ 2 0 3
 Page 1 $\rightarrow$ 2 0 1
 Page 2 $\rightarrow$ 2 0 1
 Page 0 $\rightarrow$ 2 0 1

Page 1 $\rightarrow$ 201

Page 7 $\rightarrow$ 701

Page 0 $\rightarrow$ 701

Page 1 $\rightarrow$ 701

Total Page faults = 9

STTS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS

STFS